# BCSE301L_SOFTWARE ENGINEERING

## SOFTWARE EVOLUTION

## Module 6

**Presented By,**
***Dr.Baiju B V***
***Assistant Professor***
***School of Computer Science and Engineering VIT, Vellore***

# SOFTWARE MAINTENANCE

- Software Maintenance is the ***process of modifying a software product*** after it has been delivered to the customer.
- The main purpose of software maintenance is to modify and update software application after delivery to ***correct faults*** and to ***improve performance***.

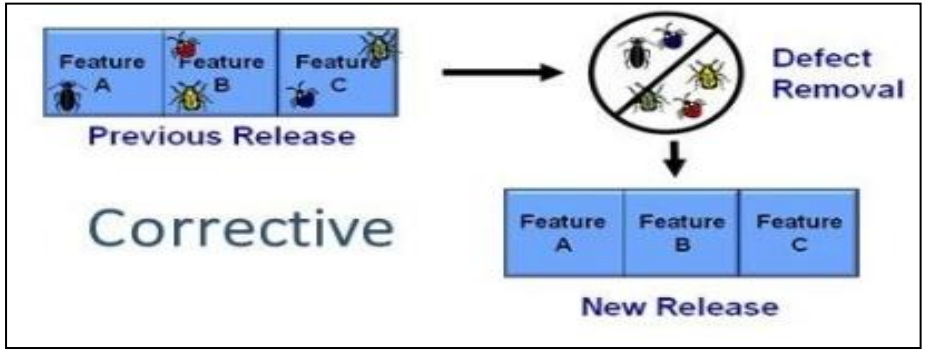
**Need for Maintenance**

- Software Maintenance must be performed in order to:
  - Correct faults
  - Improve the design
  - Implement enhancements
  - Interface with other systems
  - Make programs compatible with various hardware, software, system features, and telecom facilities.
  - Migrate legacy software.
  - Retire software.

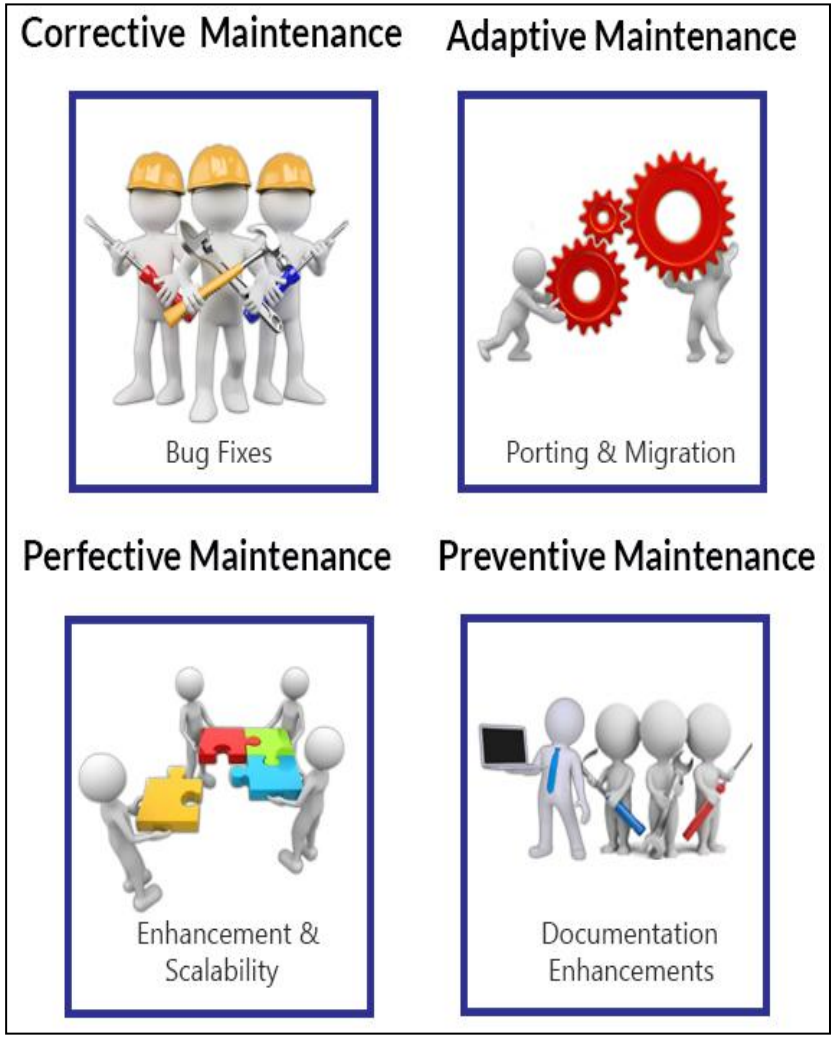
# Types of Maintenance

## 1. Corrective Maintenance

- Corrective maintenance of a software product may be essential either to ***rectify some bugs*** observed while the system is in use, or to ***enhance the performance of the system***.

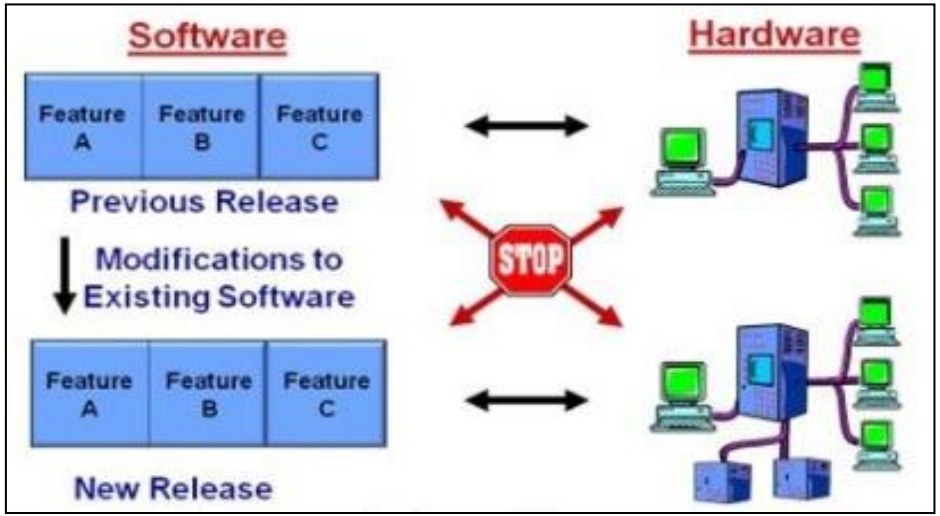


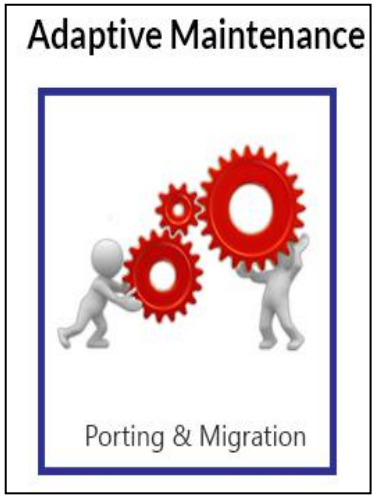

***A user reports a bug in the login module of an application where incorrect password entries are not displaying an error message. The development team fixes the bug and releases a patch.***



## 2. Adaptive Maintenance

- This includes ***modifications*** and ***updations*** when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.



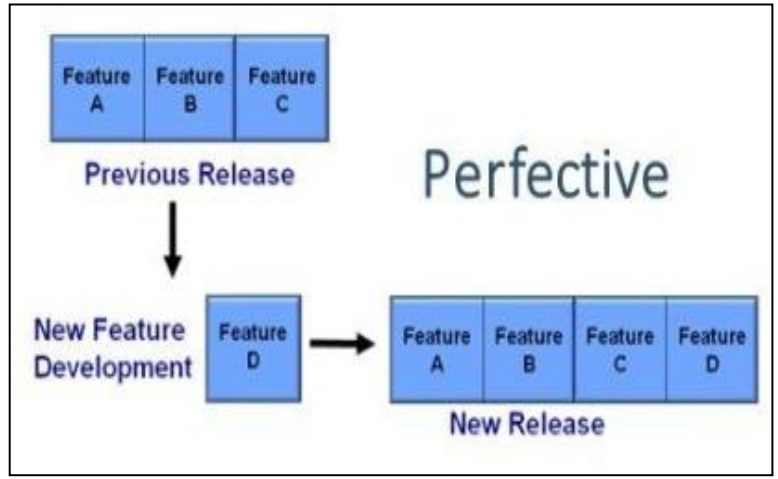




***An organization upgrades its database from MySQL to PostgreSQL. The software needs adaptive maintenance to ensure compatibility with the new database.***

***A web application is updated to run smoothly on the latest version of Chrome after a browser update.***

## 3. Perfective Maintenance

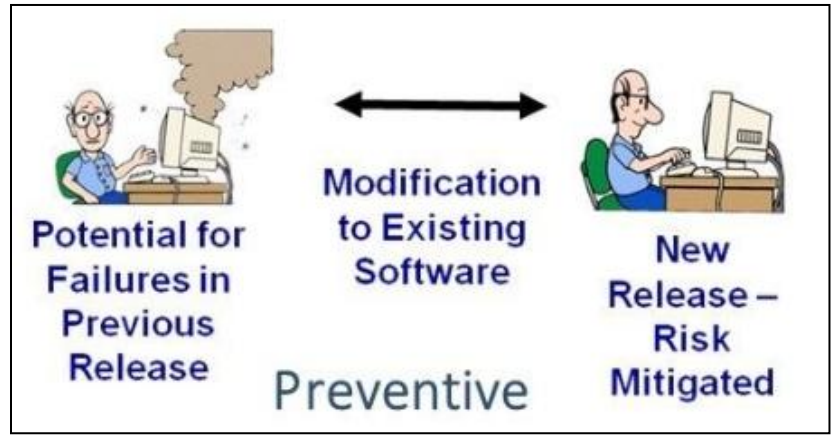
- A software product needs maintenance to ***add new features*** or ***modify functionalities*** based on customer demands.



***Adding a <u>voice search feature</u> to a shopping app for better user experience.***

## 4. Preventive Maintenance

- This type of maintenance includes modifications and updations to prevent future problems of the software.




***Refactoring code to remove unused functions and improve performance.***

# RE-ENGINEERING

- Software Re-engineering is a process of software development that is done to ***improve the maintainability of a software system***.
- Reengineering may involve
  - ***Redocumenting the system***
  - ***Refactoring the system architecture***
  - ***Translating programs to a modern programming language***
  - ***Modifying and updating the system's data and structure.***
- Reengineering has two important advantages over replacement:

**(i) Reduced Risk**

- Redeveloping essential business software is risky because errors in planning or development can cause delays, increase costs, and disrupt business operations.

**(ii) Reduced Cost**

- The cost of reengineering may be significantly less than the cost of developing new software.

- Reengineering is a rebuilding activity.

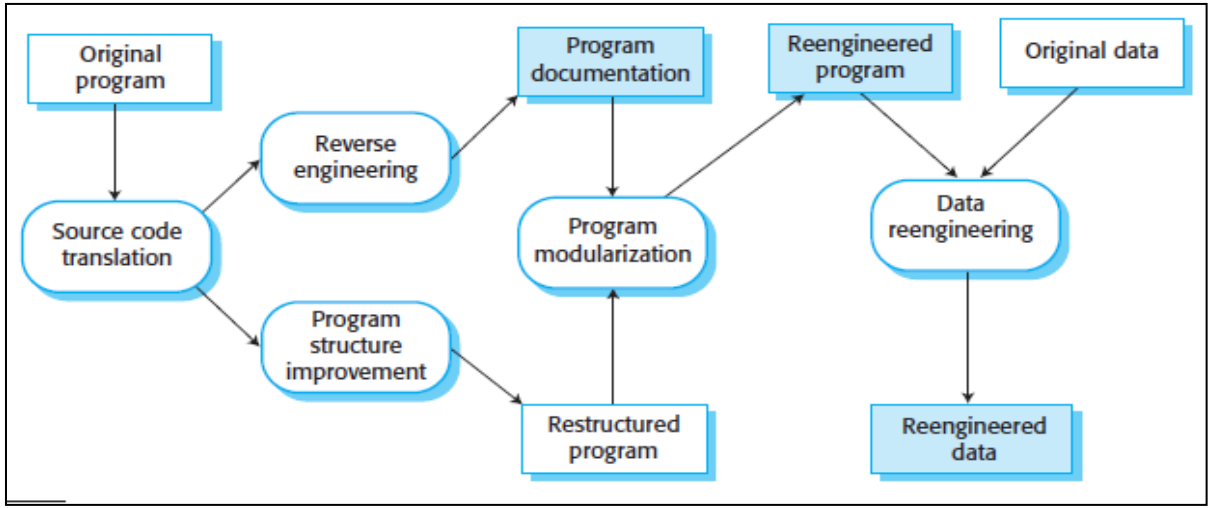
Consider an analogous activity: **Rebuilding of a house**

***You've purchased a house in another state. You've never actually seen the property, but you acquired it at an amazingly low price, with the warning that it might have to be completely rebuilt. How would you proceed?***

- Before you can start rebuilding, it would seem reasonable to ***inspect the house***. To determine whether it is in need of rebuilding, you (or a professional inspector) would create a list of criteria so that your inspection would be systematic.
- Before you tear down and rebuild the entire house, ***be sure that the structure is weak***. If the house is structurally sound, it may be possible to "remodel" without rebuilding (at much lower cost and in much less time).
- Before you start rebuilding be sure you ***understand how the original was built***. Take a peek behind the walls. Understand the wiring, the plumbing, and the structural internals.
- If you begin to rebuild, ***use only the most modern, long-lasting materials***. This may cost a bit more now, but it will help you to avoid expensive and time-consuming maintenance later.
- If you decide to rebuild, ***be disciplined*** about it. Use practices that will result in high quality—today and in the future

# Reengineering Process Model

- The **input** to the process is a ***legacy program***, and the **output** is an ***improved and restructured version*** of the same program.
- The activities in this reengineering process are:




## 1. *Source Code Translation*

- A **translation tool** can convert a program from an old programming language to a newer version or a different language.

***A COBOL-based banking application is translated into Java to make it compatible with modern platforms.***

## 2. *Reverse engineering*

- The program is analyzed and information extracted from it.
- This helps to document its organization and functionality.
- This process is usually completely automated.

***Engineers reverse-engineer a legacy C++ app to create a UML class diagram, making the system's architecture clearer.***

## 3. *Program Structure Improvement*

- Enhances the structure of the program by optimizing code, removing redundancies, and improving readability.

***Refactoring a PHP website by removing duplicated code blocks, making it more efficient and easier to maintain.***

## 4. *Program Modularization*

- Breaks the program into smaller, manageable modules for better flexibility and maintenance.

***A monolithic e-commerce platform is modularized into separate services for inventory, orders, and payments, improving scalability.***

## 5. *Restructured Program*

- The improved and modularized version of the program with better code organization.

***A content management system (CMS) is restructured with cleaner code and improved performance.***

## 6. *Data reengineering*

- Improves and optimizes the data structure for better efficiency and accessibility.

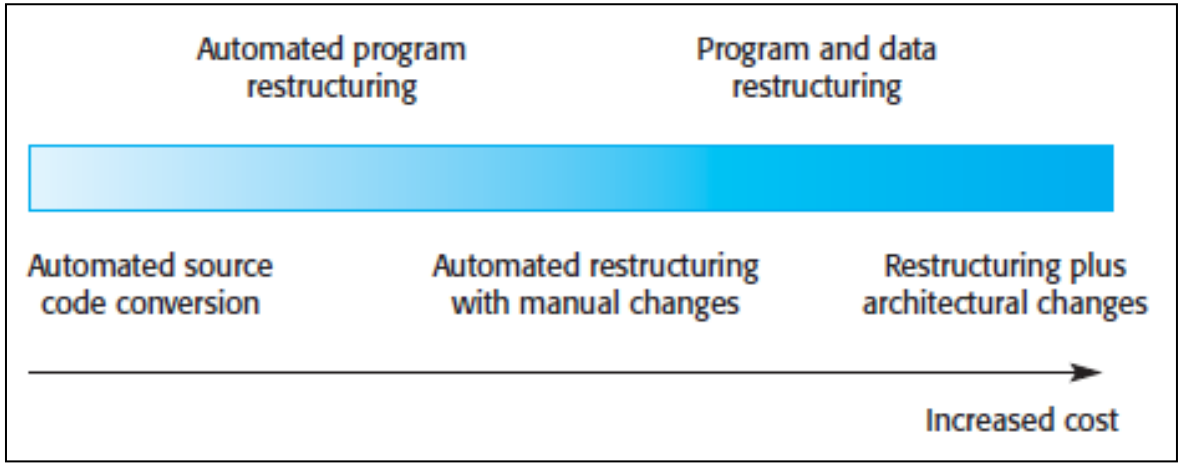
***A flat-file customer database is migrated to a SQL-based relational database for faster queries and better data management.***

## 7. *Reengineered Data*

- The final, optimized version of the data.

***The customer data in an online shopping app is cleaned and organized, making report generation faster and more efficient.***

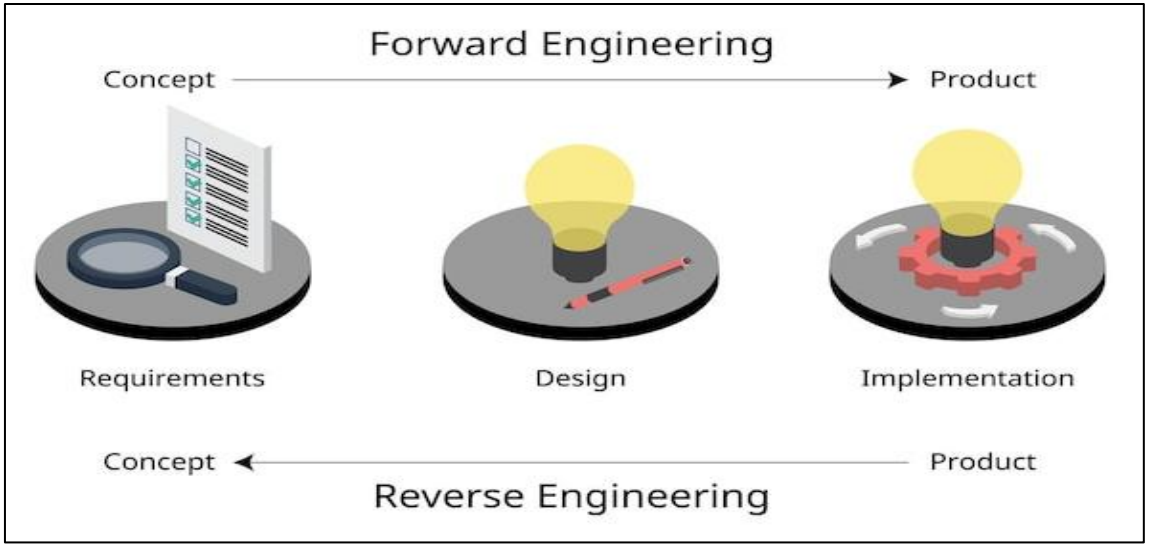
- The **costs of reengineering** obviously depend on the extent of the work that is carried out.
- There is a spectrum of possible approaches to reengineering

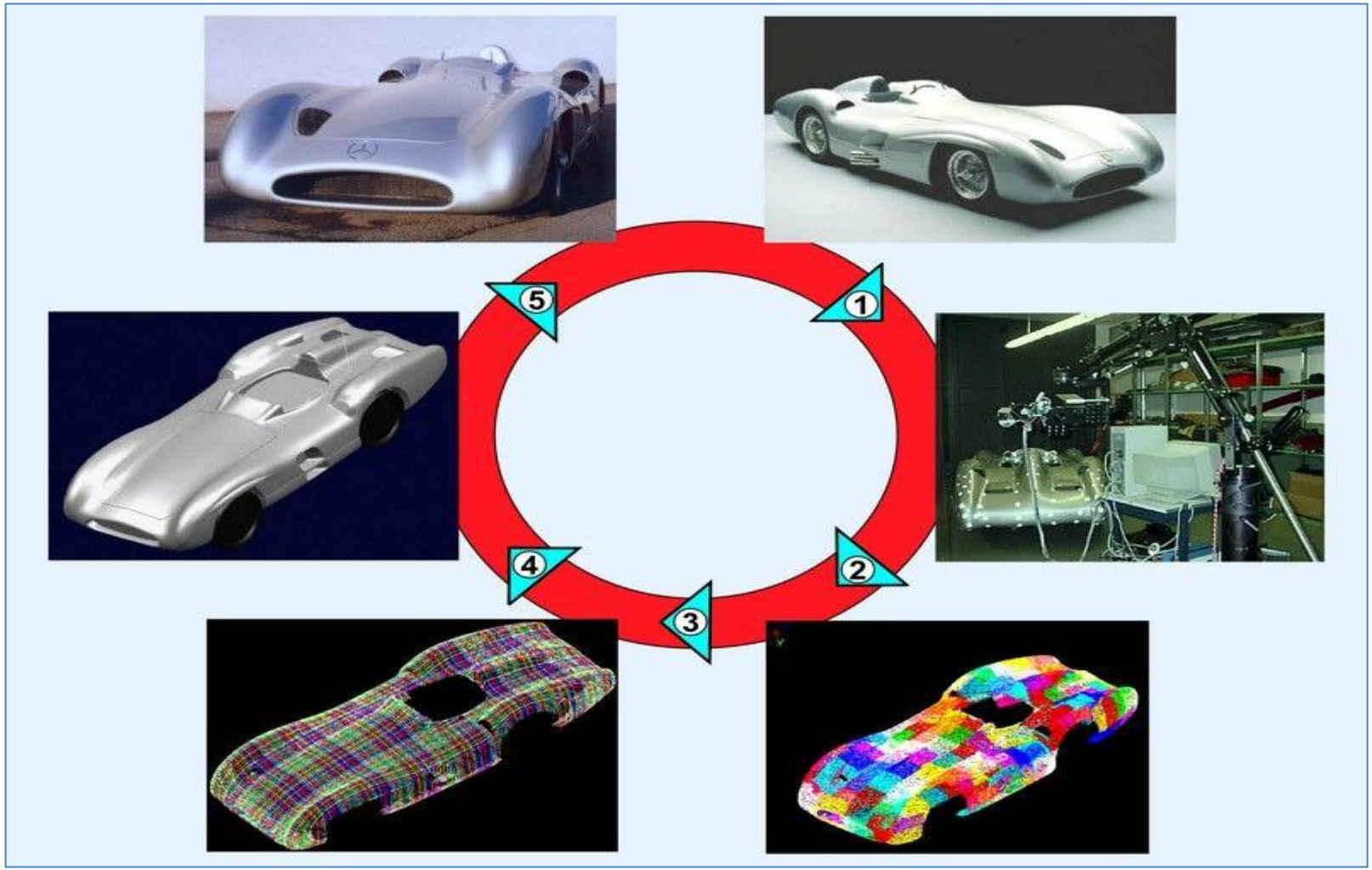



- Costs increase from left to right so that source code translation is the cheapest option, and reengineering, as part of architectural migration, is the most expensive.

# REVERSE ENGINEERING

- Reverse Engineering is ***process of extracting knowledge or design from existing software or systems and reproducing it.***
- It is also called **back Engineering**.
- Reverse engineering involves
  - ***Studying old software by analyzing its source code***
  - ***Understanding how it works***
  - ***Examining the user interface and database.***






***1. Capture the physical model → 2. Scan and digitize it → 3. Generate a point cloud or mesh → 4. Reconstruct the surface → 5. Create the final 3D model***

1. **Abstraction Level:**

- Refers to how much ***high-level design information*** can be extracted from the source code.
- Higher abstraction levels provide a better overall understanding of the program.
- Levels of abstraction (from low to high):
  - **Procedural design**: *Low-level details like individual functions.*
  - **Program and data structures**: *Shows how the data and logic are organized.*
  - ***Object models, data, and control flow models***: *Mid-level representations showing how different parts interact.*
  - ***Entity-relationship models (ER diagrams)***: *High-level overview of the system's data and relationships.*

2. **Completeness:**

- Refers to how ***detailed the extracted information*** is at each abstraction level
- As abstraction increases, completeness decreases.

***When reverse engineering a car,***

- ***At a low level, you extract detailed engine parts and wiring information (high completeness).***
- ***At a high level, you only show the car's exterior shape and major components, missing internal details (low completeness).***

## 3. Interactivity:

- Interactivity means ***how much a person is involved*** in reverse engineering along with automated tools.
- At higher abstraction levels, more human effort is needed to keep the details accurate.

***When reverse engineering a website:***

- **Low abstraction (Code level):** ***Automated tools can extract HTML, CSS, and JavaScript with minimal human involvement.***
- **High abstraction (UML diagrams):** ***The engineer needs to manually interpret the relationships between classes and modules, as tools can't fully capture the design logic.***

## 4. Directionality:

- **One-way direction**: The extracted information is used by the engineer for maintenance and documentation.

***An engineer extracts the code structure from an old payroll system to document it for maintenance, but no changes are made to the original program.***

- **Two-way direction:** The extracted details are sent to a reengineering tool that modifies or regenerates the program.

***The extracted code is fed into a reengineering tool, which modifies and regenerates the program with improved performance or updated features.***

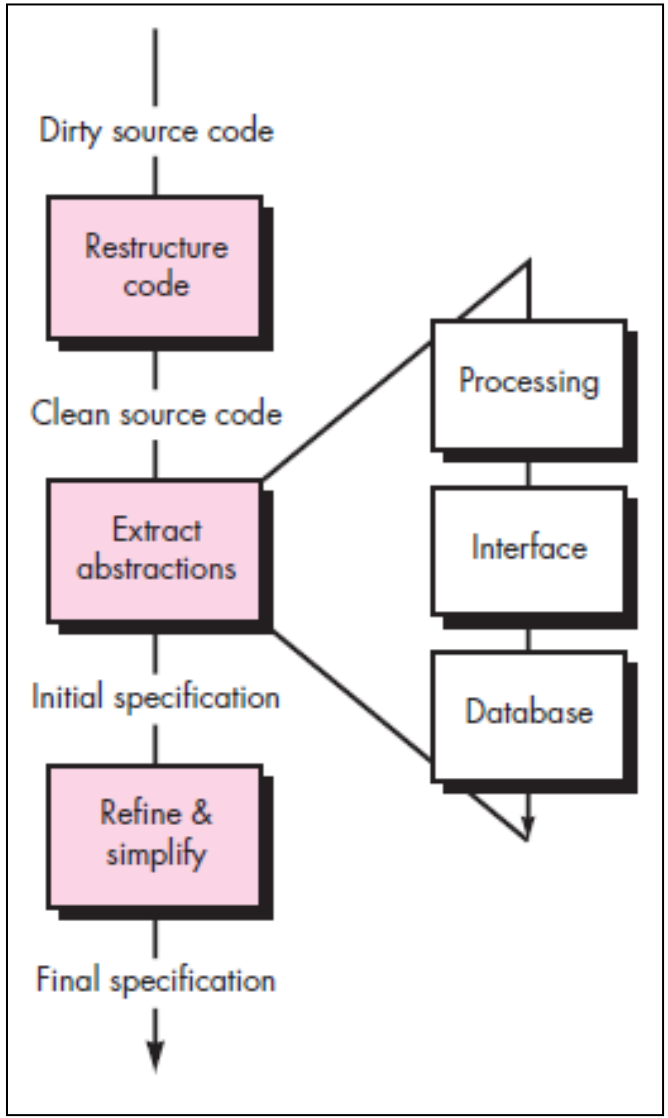
## Reverse Engineering Process

**Code Restructuring:**

- Before starting reverse engineering, unstructured (messy) source code is cleaned up.
- It is converted into structured code using proper programming constructs.
- This makes the code easier to read and serves as a foundation for further reverse engineering steps.

**Extracting Abstractions:**

- The main task in reverse engineering is extracting abstractions.
- This means identifying higher-level design details (like structure, flow, or models) from the source code.



## 1. Reverse Engineering to Understand Data

- Reverse engineering happens at **different levels of abstraction** and is usually the first step in reengineering.
  - ***Program level*:** It involves **analyzing internal program** data structures.
  - ***System level*:** It focuses on **reengineering global data structures** (like files or databases) to **support new database models** (e.g., shifting from flat files to relational or object-oriented databases).
  - ***UML diagrams*:** Sometimes, the first reverse engineering task is to create a UML class diagram.

### a. Internal Data Structures

- Reverse engineering for internal program data focuses on **identifying object classes**.
  - This is done by analyzing the code to group related variables.
  - The way data is organized in the code often points to abstract data types.

***When reverse engineering a billing system, analyzing the code may reveal related variables like CustomerName, Address, and PhoneNumber, which can be grouped into a Customer class.***

## b. Database Structure

- Reengineering a database involves ***understanding the existing data objects and their relationships***.
- The steps to define the current data model before creating a new one are:
  - ✓ **Build an initial object model:** Identify the main data objects.
  - ✓ **Determine candidate keys:** Find unique identifiers for each object.
  - ✓ **Refine tentative classes:** Improve and organize the object classes.
  - ✓ **Define generalizations:** Group similar objects into broader categories.
  - ✓ **Discover associations:** Identify relationships between objects using techniques similar to **CRC (Class-Responsibility-Collaborator)** approach.

***When reengineering a <u>library</u> database:***

- **Initial object model:** ***Identify objects like Books, Members, and Loans.***
- **Candidate keys**: ***Use BookID and MemberID as unique identifiers.***
- **Refine classes**: ***Organize data into Book, Member, and Loan classes.***
- **Generalizations**: ***Group Books and Magazines under a Publication category.***
- **Associations**: ***Link Members to Loans and Books to Loans.***

## 2. Reverse Engineering to Understand Processing

- Reverse engineering starts by ***understanding and extracting procedural abstractions*** from the source code.
  - ✓ **Code analysis** is done at different levels:
    - System
    - Program
    - Component
    - Pattern
    - Statement.
  - ✓ **Overall application functionality** is understood first to identify how different programs interact.
  - ✓ **Block diagram** is created to show how these programs (functional abstractions) interact.
  - ✓ Each **component** performs a specific task and represents a procedural abstraction.
  - ✓ A **processing narrative** is written to describe how each component works.

## 3. Reverse Engineering to Understand User Interfaces

- **Redeveloping user interfaces** is a common reengineering task.
- Before rebuilding, reverse engineering is needed to understand the current interface's structure and behavior.
- Three key questions to answer during UI reverse engineering:
  - ***What actions (e.g., keystrokes, mouse clicks) does the interface handle?***
  - ***How does the system respond to these actions?***
  - ***What defines a "replacement" or an equivalent interface?***

**When redesigning a mobile app, reverse engineering helps understand the current interface.**

- **Actions**: Tapping buttons or swiping screens.
- **System response**: Displaying a new page or updating data.
- **Replacement**: The new interface should offer the same functions but with a better design.

# SOFTWARE REUSE

- Reuse-based software engineering is an approach to development that tries to ***maximize the reuse of existing software***.
- The software units that are reused may be of radically different sizes.

**a. System Reuse :** *Reusing an entire system as part of a larger system.*

**A hospital management system (HMS) used in one hospital can be reused by another hospital as part of a multi-hospital network.**

**b. Application Reuse :** *Reusing a complete application in another system, either directly or with minor configuration changes.*

**A library management application used in one school can be reused in another school with slight modifications.**

**c. Component Reuse :** *Reusing parts of an application, like subsystems or modules.*

**A user authentication module from an e-commerce app can be reused in a banking app**

**d. Object and Function Reuse :** *Reusing smaller software units, such as classes or functions.*

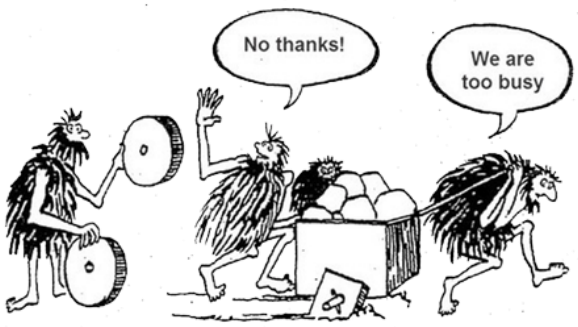
**A function to calculate discount in an online shopping app can be reused in a restaurant billing system.**

## Benefits of Software Reuse

| Benefit | Explanation |
|---|---|
| ***Increased Dependability*** | • Reused software is more reliable than new software because its flaws have already been identified and fixed. |
| ***Reduced Process Risk*** | • The cost of existing software is clear, but development costs are harder to predict.<br>• This helps project management reduce errors in cost estimation. |
| ***Effective use of Specialists*** | • Instead of repeating the same work, application specialists can create reusable software that captures their expertise.<br>• A developer creating a login module can make it reusable, so it can be used in multiple apps without rewriting the code. |
| ***Standards Compliance*** | • User interface standards can use reusable components.<br>• For example, if all menus use the same component, every application shows a consistent menu format. |
| ***Accelerated development*** | • Launching a system quickly is often more important than its development cost.<br>• Reusing software speeds up production by reducing both development and testing time. |

# Problems with Software Reuse

| Problem | Explanation |
|---|---|
| ***Increased Maintenance Costs*** | • If the source code of reused software is unavailable, maintenance costs may increase because the reused parts might not work well with system changes. |
| ***Lack of tool support*** | • Some software tools don't support reuse, making it hard to integrate with component libraries.<br>• This is more common in embedded systems than in object-oriented tools. |
| ***Not-invented-here syndrome*** | • Some software engineers prefer to rewrite components because they believe they can improve on them.<br>• This is partly about trust and partly because creating original software is harder than reusing existing software. |

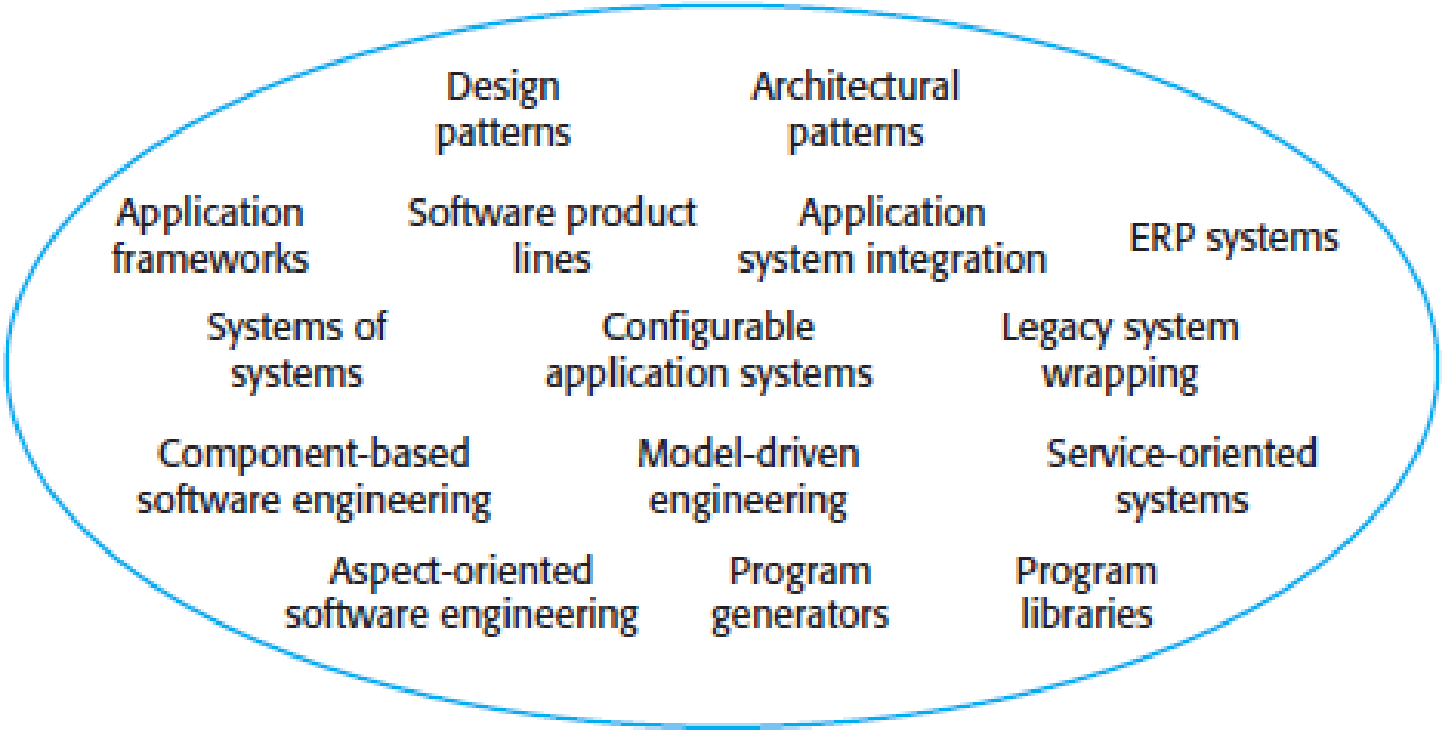


## Problems with Software Reuse

| Problem | Explanation |
|---|---|
| ***Creating, Maintaining, and using a component library*** | • Building a reusable component library and making sure developers use it can be costly.<br>• The development process needs to be adjusted to include the library. |
| ***Finding, Understanding, and Adapting reusable components*** | • Engineers need to find, understand, and sometimes modify software components from a library to fit a new environment.<br>• They should be fairly sure of finding a suitable component before adding the search to their development process.<br>***When building a website, engineers might look for a ready-made login module in a library. They need to understand and possibly tweak it to fit their website.*** |

## 1. The reuse landscape

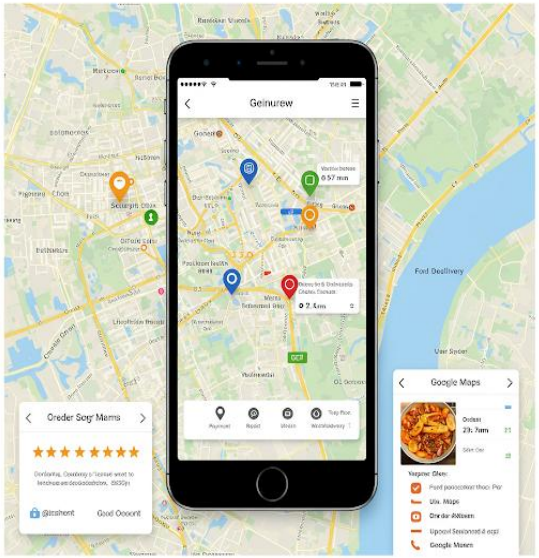
- Reuse is not just about reusing system components.
- It can involve anything from small functions to entire applications, using different reuse methods.



# Approaches that support software reuse

| Approach | Description |
|---|---|
| **Architectural Patterns** | • **Standard software designs** are used as a base for building common types of applications.<br>***E-commerce websites often use standard shopping cart designs as a base.*** |
| **Design Patterns** | • Represent common solutions used in different applications, showing how abstract and concrete objects interact.<br>***A shopping app and a food delivery app both use a payment module, showing how abstract (payment interface) and concrete (credit card, UPI) objects interact.*** |
| **Component-Based Development** | • Systems are built by combining components that follow common standards.<br>***A shopping website is built by combining payment, cart, and user login components that follow common standards.*** |

| Approach | Description |
| --- | --- |
| **Application Frameworks** | • Abstract and concrete classes are modified and expanded to build application systems.<br>***A car class can be expanded with engine and wheel classes to create a car simulation system.*** |
| **Legacy System Wrapping** | • Legacy systems are given new interfaces to make them accessible.<br>***An old billing system is given a new web interface for online access.*** |
| **Service-Oriented Systems** | • Systems are built by connecting shared services, which can come from outside sources.<br>***A food delivery app links to Google Maps for location services.*** |
| **Software Product Lines** | • An application is built with a common design so it can be customized for different customers.<br>***A billing software can be customized for different shops by changing the product list.*** |



| Approach | Description |
| --- | --- |
| **COTS Product Reuse** | • Systems are built by combining and setting up existing applications.<br>***A school management system can be built by combining attendance, grading, and library modules.*** |
| **ERP Systems** | • Large systems with general business features are customized for a specific organization.<br>***An HR software is customized for a company to manage its employee records and payroll.*** |
| **Configurable Vertical Applications** | • Generic systems are built to be customized for different customers' needs.<br>***Generic systems are built to be customized for different customers' needs.*** |
| **Program Libraries** | • Libraries with ready-made classes and functions can be reused for common tasks.<br>***A math library with functions for addition and multiplication can be reused in different programs.*** |
| **Model-Driven Engineering** | • Software is designed using models, and code is automatically generated from them.<br>***A login form model generates code for a user login system.*** |

| Approach | Description |
| --- | --- |
| **Program Generators** | • A generator system creates applications for a specific domain using a model provided by the user.<br>***A website builder generates a website using a user-provided template.*** |
| **Aspect-Oriented Software Development** | • Shared components are added to the program at different points during compilation.<br>***A math library is added to a calculator app during compilation.*** |

- Key factors that should be considered when planning reuse are:

**(i) The development schedule for the software**

- To develop software quickly, reusing entire systems is better than individual components.
- It may not fit perfectly but reduces development effort.

**(ii) The expected software lifetime**

- For long-lasting systems, focus on making them easy to maintain, not just on the short-term benefits of reuse.

***(iii)* The background, skills and experience of the development team**

- Reuse technologies are complex and take time to learn.
- It's best to focus on areas where your team has expertise.

***(iv) The criticality of the software and its non-functional requirements***

- For a critical system that needs external certification, proving its safety or security is hard without the source code.
- If the software needs high performance, model-driven engineering (MDE) may not work.

**(v) The application domain**

- In fields like manufacturing and medical systems, generic products can be reused by adjusting them for local needs.
- This is usually cheaper than creating a new system.

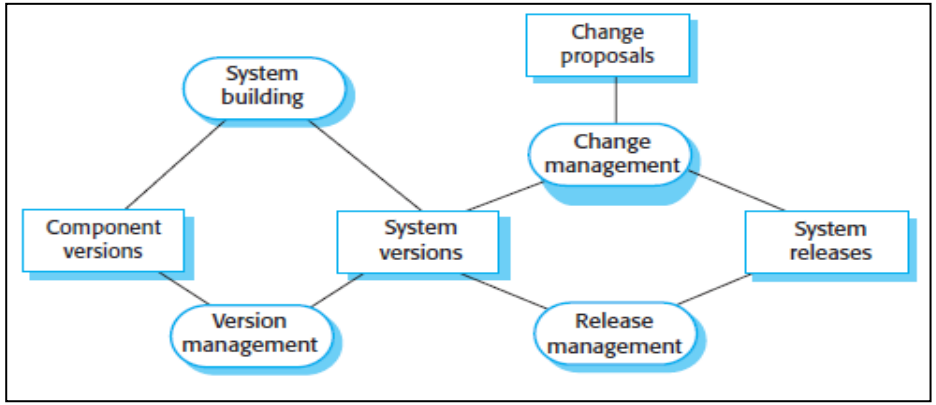
**(vi) The platform on which the system will run**

- Some component models, like .NET, only work on Microsoft platforms.
- You can reuse them only if your system is also designed for Microsoft.

# Software Configuration Management

- Configuration management (CM) is concerned with the ***policies, processes, and tools for managing changing software systems.***
- Configuration management is useful for individual projects as it is easy for one person to forget what changes have been made.
- It is essential for team projects where several developers are working at the same time on a software system.
- Configuration management tools are used to
  - ***Store versions of system components***
  - ***Build systems from these components***
  - ***Track the releases of system versions to customers***
  - ***Keep track of change proposals***
- CM tools range from simple tools that support a single configuration management task, such as bug tracking, to integrated environments that support all configuration management activities.

- The configuration management of a software system product involves four closely related activities




## 1. Version Control

- This involves ***keeping track of the multiple versions of system components*** and ensuring that changes made to components by different developers do not interfere with each other.

- Imagine you are working on a website project.
- You create multiple versions of the homepage:
  - ✓ **homepage_v1.html (Initial version)**
  - ✓ **homepage_v2.html (Updated with new images)**
  - ✓ **homepage_v3.html (Fixed a typo and improved layout)**

## 2. System Building

- This is the process of ***assembling*** program components, data, and libraries, then ***compiling*** and ***linking*** these to create an executable system.

- In a Java project, you write several .java files (modules).
  - ✓ **Compiling**: The .java files are compiled into .class files.
  - ✓ **Linking**: The compiled files are combined into a single .jar file.
  - ✓ This complete .jar file represents the built system, ready for execution.

## 3. Change Management

- It means ***tracking change requests*** from customers and developers, ***estimating the cost and impact***, and deciding if and when to implement them.

> While developing a mobile app, a client requests a new feature: **dark mode**.
>
> - ✓ ***You create a change request.***
> - ✓ ***The development team evaluates the cost, effort, and potential impact.***
> - ✓ ***Once approved, you implement the feature and merge it into the main codebase.***

## 4. Release Management

- This involves ***preparing software for external release*** and ***keeping track of the system versions*** that have been released for customer use.

> After thorough testing, you prepare your app for release on the Google Play Store.
>
> - ✓ ***The release management process involves creating a versioned package (e.g., v1.0.1), uploading it to the store, and publishing it.***
> - ✓ ***Users can now download the updated version.***

The development of a software product or custom software system takes place in three distinct phases:

**1. *A Development Phase***

- The development team is ***responsible for managing the software configuration*** and ***new functionality*** is being added to the software.
- The development team decides on the changes to be made to the system.

***2. A System Testing Phase***

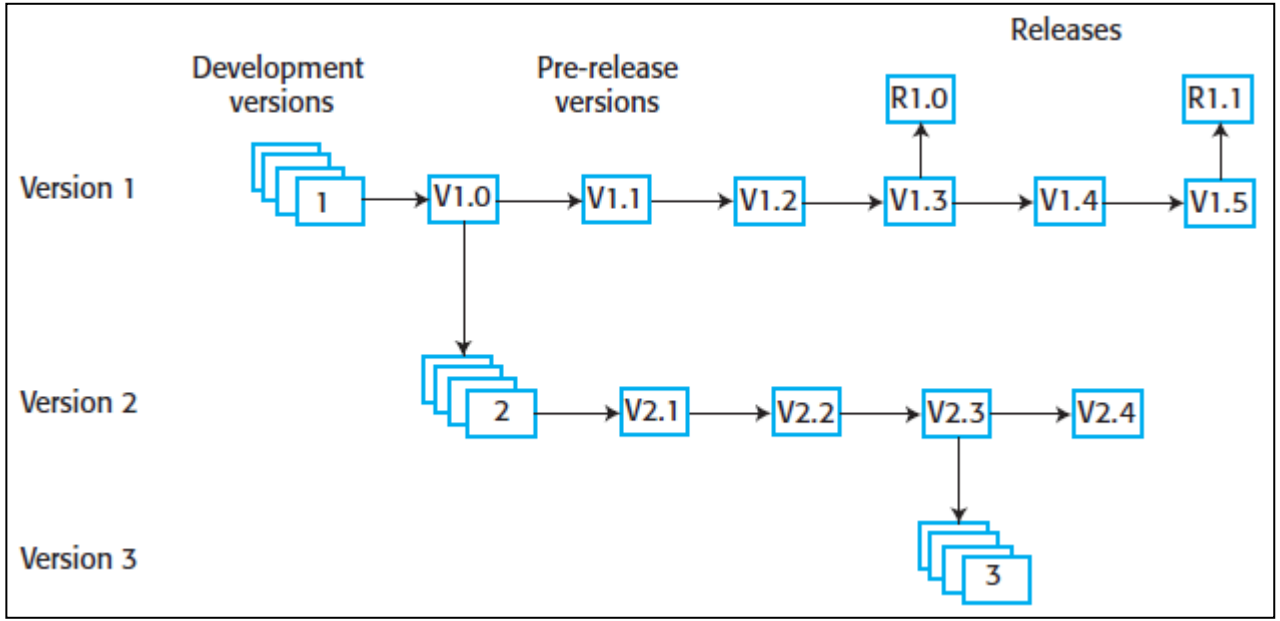
- A version of the system is ***released internally for testing***.
- This may be the responsibility of a quality management team or an individual or group within the development team.
- At this stage, ***no new functionality is added to the system***.
- The changes made at this stage are ***bug fixes***, ***performance improvements***, and ***security vulnerability repairs***.

**3. *A Release Phase***

- The software is ***released to customers for use***.
- After the release has been distributed, customers may submit bug reports and change requests.
- New versions of the released system may be developed to repair bugs and vulnerabilities and to include new features suggested by customers.

- For **large systems**, there is never just one "working" version of a system; there are always several versions of the system at different stages of development.

- **Version 1.5** of the system has been ***developed to repair bug fixes and improve the performance*** of the first release of the system.
- It is the basis of the second system release (R1.1).




- **Version 2.4** is being ***tested*** with a view to it becoming release 2.0 of the system. No new features are being added at this stage.
- **Version 3** is a ***development system*** where new features are being added in response to change requests from customers and the development team. This will eventually be released as release 3.0.

# Simple Example: Word Processing Software

## Version 1:

### Development Phase:

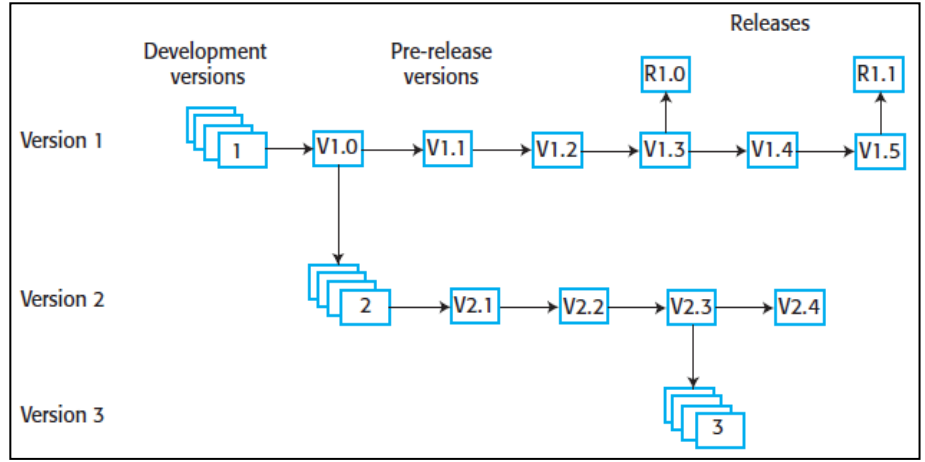
- ✓ The team builds the basic version with features like ***text editing*** and ***saving files***.

### Pre-release Phase:

- ✓ V1.0: Internal testing with basic editing features.
- ✓ V1.1: Bug fixes and minor UI improvements.
- ✓ V1.2: Added spell check.
- ✓ V1.3: Final pre-release with stability fixes.

### Release Phase:

- ✓ R1.0: Official release with text editing and spell check.
- ✓ R1.1: Minor release with bug fixes.




## Version 2:

### Development Phase:

- ✓ The team adds ***formatting options*** like bold, italic, and underline.

### •Pre-release Phase:

- ✓ V2.1: Pre-release testing with formatting features.
- ✓ V2.2: Fixed alignment issues.
- ✓ V2.3: Added page numbering.
- ✓ V2.4: Final pre-release with all fixes.

## Version 3:

### Development Phase:

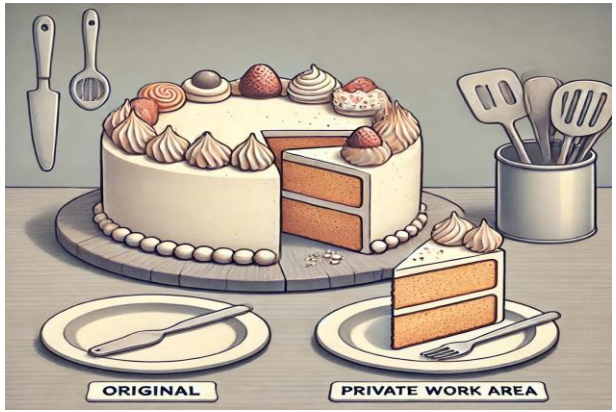
- ✓ The team begins working on collaboration features (e.g., real-time editing), but it is still under development.

# CM Terminology

| Term | Explanation |
|---|---|
| **Baseline** | • A baseline is a collection of component versions that form a system.<br>• It is fixed and cannot be changed once created.<br>• The system can always be recreated using the same component versions in the baseline. |
| **Branching** | • The creation of a new codeline from a version in an existing codeline.<br>• The new codeline and the existing codeline may then develop independently. |
| **Codeline** | • A set of software component versions and their dependent configuration items. |
| **Configuration (version) control** | • The process of keeping track of system and component versions to manage changes and store all versions throughout the system's lifetime. |

| Term | Explanation |
| --- | --- |
| **Mainline** | • A sequence of baselines representing different versions of a system |
| **Merging** | • Creating a new version of a software component by combining different versions from separate codelines.<br>• These codelines may have originally branched from one of the versions involved.<br>**Imagine you are working on a mobile app with two separate codelines:**<br>• ***Codeline A: Adds a new login feature.***<br>• ***Codeline B: Improves the UI design.***<br>**Both codelines originally branched from the main version. To create a new version, you merge the changes from both codelines into a single version, combining the new login feature with the improved UI design.** |
| **Release** | • A version of a system that has been released to customers (or other users in an organization) for use. |

| Term | Explanation |
|---|---|
| **Repository** | • Shared database that stores different versions of software components along with details about the changes made to them. |
| **System building** | • Creating a working software version by compiling and linking the right components and libraries together. |
| **Version** | • A version is a specific instance of a software component that is different from other instances.<br>• Each version has a unique identifier to distinguish it. |
| **Workspace** | • A private work area where software can be modified without affecting other developers who may be using or modifying that software. |



## 1. VERSION MANAGEMENT

- Version management is the ***process of keeping track of different versions of software components and the systems*** in which these components are used.
- It also involves ensuring that changes made by different developers to these versions do not interfere with each other.
- Version management is the process of ***managing codelines and baselines***.

***Codeline***

- A codeline is a ***series of source code versions***, where each new version is based on a previous one.
- It is used for system components, creating different versions of each part.

***Baseline***

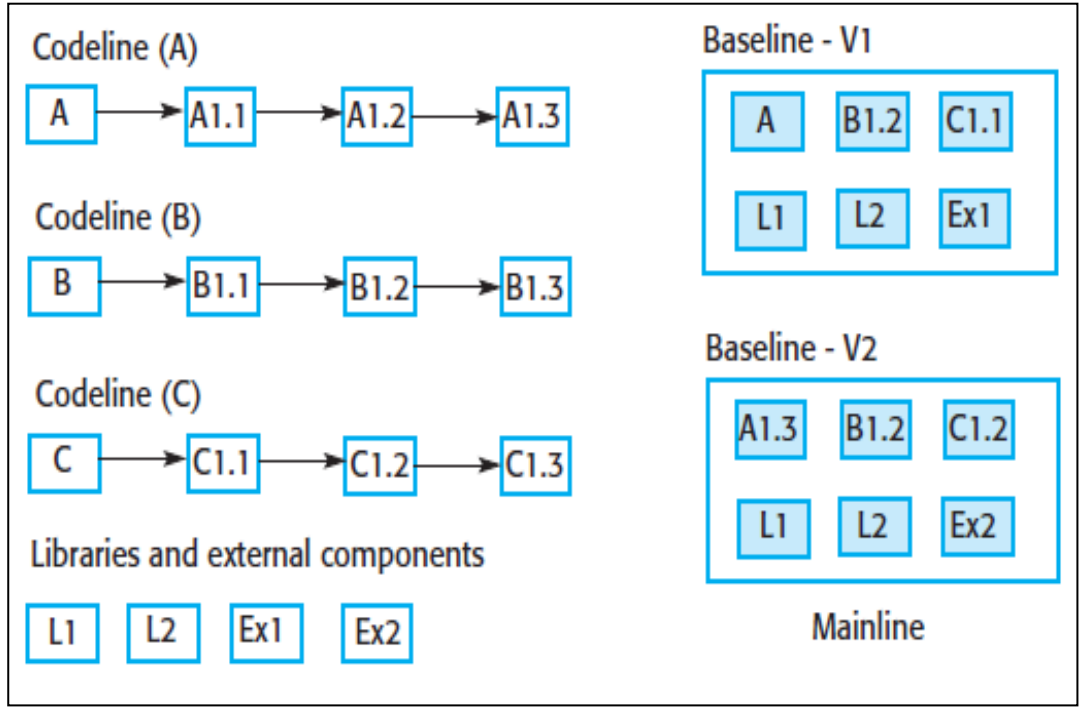
- A baseline defines a ***specific version of a system***..
- It defines the ***component versions***, ***libraries***, ***configuration files***, and other system details included in that version.

***Mainline***

- The mainline is ***a series of system versions*** created from the original baseline.

In a car manufacturing project,

- **Engine module** (**Codeline A**) evolves from basic engine (A1.1) to fuel-efficient engine (A1.3).
- **Dashboard module** (**Codeline B**) progresses from basic display (B1.1) to an enhanced touchscreen (B1.3).




- **Baseline V1** could represent a first release of the car, using older engine and dashboard versions.
- **Baseline V2** could be an upgraded release, incorporating the latest engine version (A1.3) and external component (Ex2).

- **Mainline** shows the continuous evolution from Baseline V1 (first release) to Baseline V2 (updated release).
- Future versions (V3, V4, etc.) will continue to be added to the mainline.

## Version Control (VC) Systems

- Identify, store, and control access to the different versions of components.
- There are two types of **modern version control system**:
  1. ***Centralized systems*** **:**
     - A single main repository stores all versions of the software.
     - Developers access and update it from one central location.
     - **Google Docs – Everyone edits the same online document stored on a central server.**
  2. ***Distributed systems*** :
     - Multiple copies of the repository exist on different systems.
     - **Git – Developers work on local copies of the code and later merge their changes with a shared repository.**
- Key features of these systems include:

**1. Version and Release Identification**

- Managed versions of a component are assigned **unique identifiers** when they are submitted to the system.
- This ID helps manage different versions of the same component without changing its name.

## 2. Change History Recording

- A Version Control System (VCS) **keeps track of all the changes** made to a component over time.
  - ✓ It tags changes with keywords (e.g., "bug fix" or "new feature").
  - ✓ You can use these tags to find and include specific versions in a baseline.

***If you update a login feature, you tag the change as "Login Bug Fix". Later, you can search for this tag to include it in a stable version.***

## 3. Independent Development

- Multiple developers can work on the same component at the same time.
  - ✓ The VCS tracks who is editing which component.
  - ✓ It ensures that changes from different developers don't conflict.

***Two developers are editing the checkout module of an app.***

- ***The VCS prevents them from accidentally overwriting each other's work by keeping separate versions.***

## 4. Project support

- A VCS helps manage multiple projects that share common components.
  - ✓ You can check in or check out all files related to a project at once.

***If two apps use the same payment gateway, you can update the gateway once and apply the change to both apps simultaneously.***

- Software development is usually done in teams, with multiple members working on the same component at the same time.

***Let's say Alice is making some changes to a system, which involves changing components A, B, and C. At the same time, Bob is working on changes that require making changes to components X, Y, and C. Both Alice and Bob are therefore changing C. It's important to avoid situations where changes interfere with each other—Bob's changes to C overwriting Alice's or vice versa.***

Version control systems use two main areas:

**1.Project Repository:**

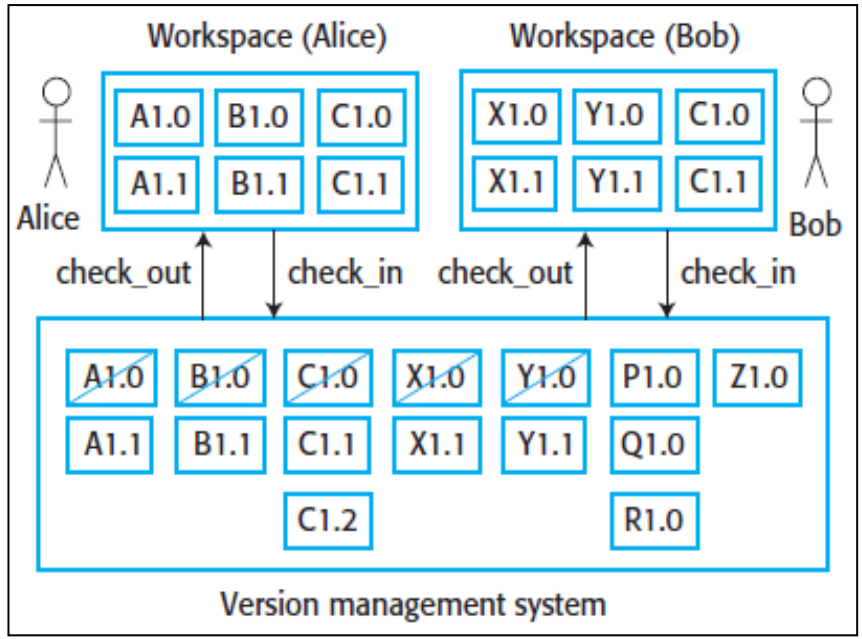
1. Stores the ***master version*** of all components.
2. Used to create baselines for building the system.

**2.Private Workspace:**

1. Developers **check out** (copy) components from the repository.
2. They make changes in their own workspace without affecting others.
3. Once done, they **check in** (return) the modified components to the repository.

- In **centralized systems**, developers copy components from a shared repository to their private workspace.
  - ✓ They make changes to the copied components.
  - ✓ Once done, they check-in the updated components back into the repository.
  - ✓ This creates a new version that others can access.

- Alice checks out versions A1.0, B1.0, and C1.0 from the repository.
- She makes changes and creates new versions: A1.1, B1.1, and C1.1.
- She checks these updated versions back into the repository.
- Bob checks out versions X1.0, Y1.0, and C1.0.
- He makes changes and creates new versions of these components.
- He also checks his updated versions into the repository.




- Alice creates a new version of **C** while Bob is still working on it.
  - ✓ When Bob checks in his changes, a new version C1.2 is created.
  - ✓ This ensures that Alice's changes are not overwritten.

- In a **distributed VC system**, such as Git, a different approach is used.

**Master Repository:**

- A main repository is stored on a central server, holding the project code.

**Cloning the Repository:**

- Instead of just checking out files, developers clone the entire repository to their own computers.
- This creates a local copy of the project, including its full history.

**Local Changes:**

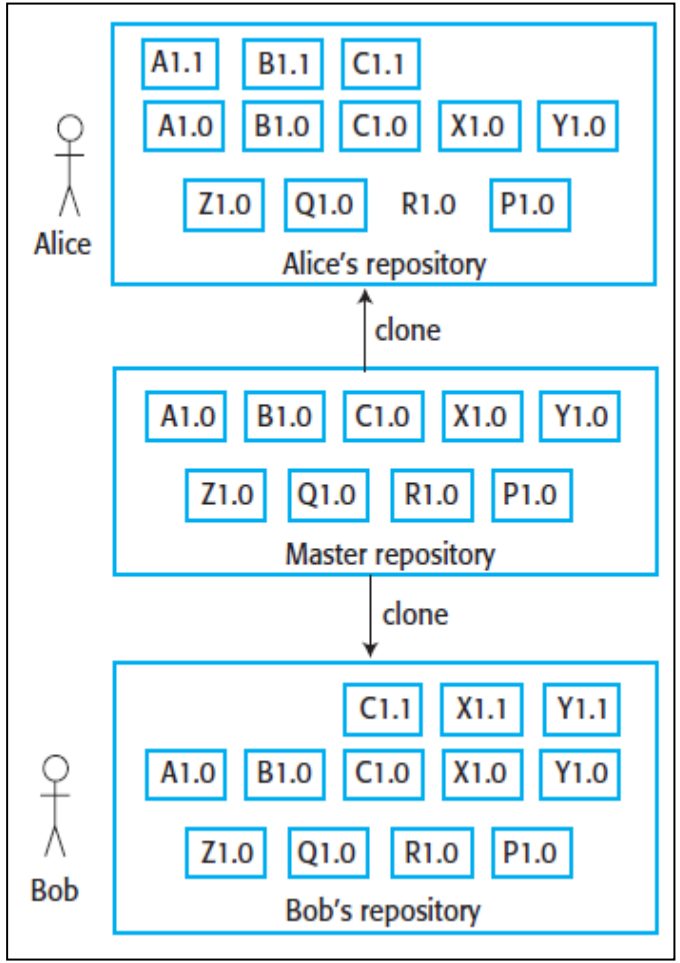
- Developers work on their local copies, make changes, and save them in their private repository on their own machine.

**Committing Changes**:

- When they finish making changes, they commit them locally, which records the changes in their private repository.

**Pushing to the Master:**

- Developers can push their changes to the master repository on the server.
- Alternatively, they can notify the integration manager to review and merge the changes into the master repository.



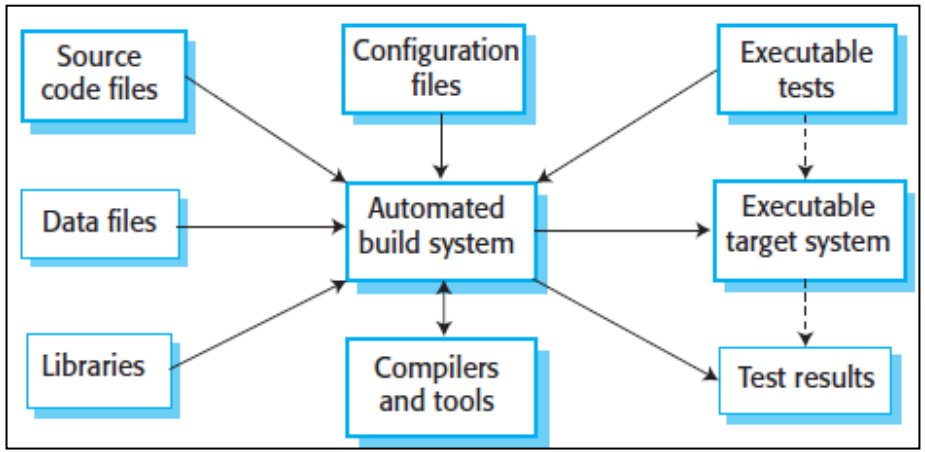

Both Bob and Alice have cloned the project repository and have updated files.

## 2. SYSTEM BUILDING

- System building is the process of creating a working software system by:
  - ✓ **Compiling and linking** system components.
  - ✓ Including **external libraries** and **configuration files**.
- System-building tools work with **version control systems** to get the correct component versions from the repository.
- It also gathers details about the software and its operating environment.

> When creating a mobile app, the build process:
> - Combines the code files, images, and database settings.
> - Uses the latest version of each file from the version control system.
> - Generates the final app file that users can run.

- An automated build tool creates the entire system.
  - ❖ You can build the system with one command or one click.



- Tools for system integration and building include some features

| Feature | Description | Example |
|---|---|---|
| ***Build Script Generation*** | The build system should:<br>• Analyze the program to find its dependent components.<br>• Automatically create a build script with the necessary steps.<br>• Allow manual editing of the build script if needed | When building a website, the system:<br>• Finds dependencies like CSS files and JavaScript libraries.<br>• Automatically creates a script to bundle them.<br>• You can still manually edit the script if you want to add or change something. |
| ***Version Control System Integration*** | • The build system should get the needed component versions from the version control system. | When building a weather app,<br>• Gets UI component(v1.2) & API module (v2.1) from the VC system.<br>• It uses these specific versions to create the final app. |
| ***Minimal Recompilation*** | Build system checks which parts of the source code have changed.<br>• It recompiles only the modified code instead of the entire program.<br>• This saves time and speeds up the build process. | You are making a school timetable in Excel.<br>• If you change Monday's schedule, you don't need to redo the entire timetable.<br>• You only update Monday's section.<br>• Similarly, the build system recompiles only the modified code, not the whole program. |

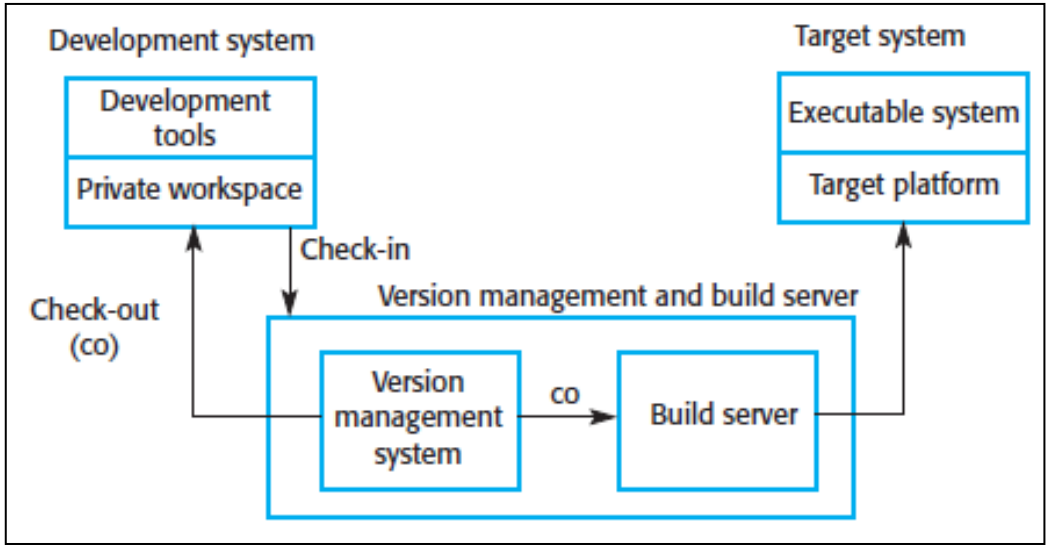
- Tools for system integration and building include some or all of the following features

| Feature | Description |
|---|---|
| ***Executable System Creation*** | • The build system should link the compiled object code files with each other and with other required files, such as libraries and configuration files, to create an executable system. |
| ***Test Automation*** | • Some build systems can automatically run automated tests using test automation tools such as JUnit. |
| ***Reporting*** | • The build system should provide ***reports about the success or failure of the build and the tests*** that have been run. |
| ***Documentation Generation*** | • The build system may be able to generate release notes about the build and system help pages |

Imagine you are building a calculator app:

The build system:
- Compiles the source code into an .exe file.
- Runs tests to verify if the calculations are correct.
- Generates reports on whether the build and tests succeeded or failed.
- Creates documentation explaining how to use the calculator's functions.

- **Building** is a complex process in which three different system platforms may be involved




## 1. The Development System

- Includes tools like **compilers** and **code editors**.
- Developers copy the code from the version control system into their own workspace to make changes

## 2. The Build Server

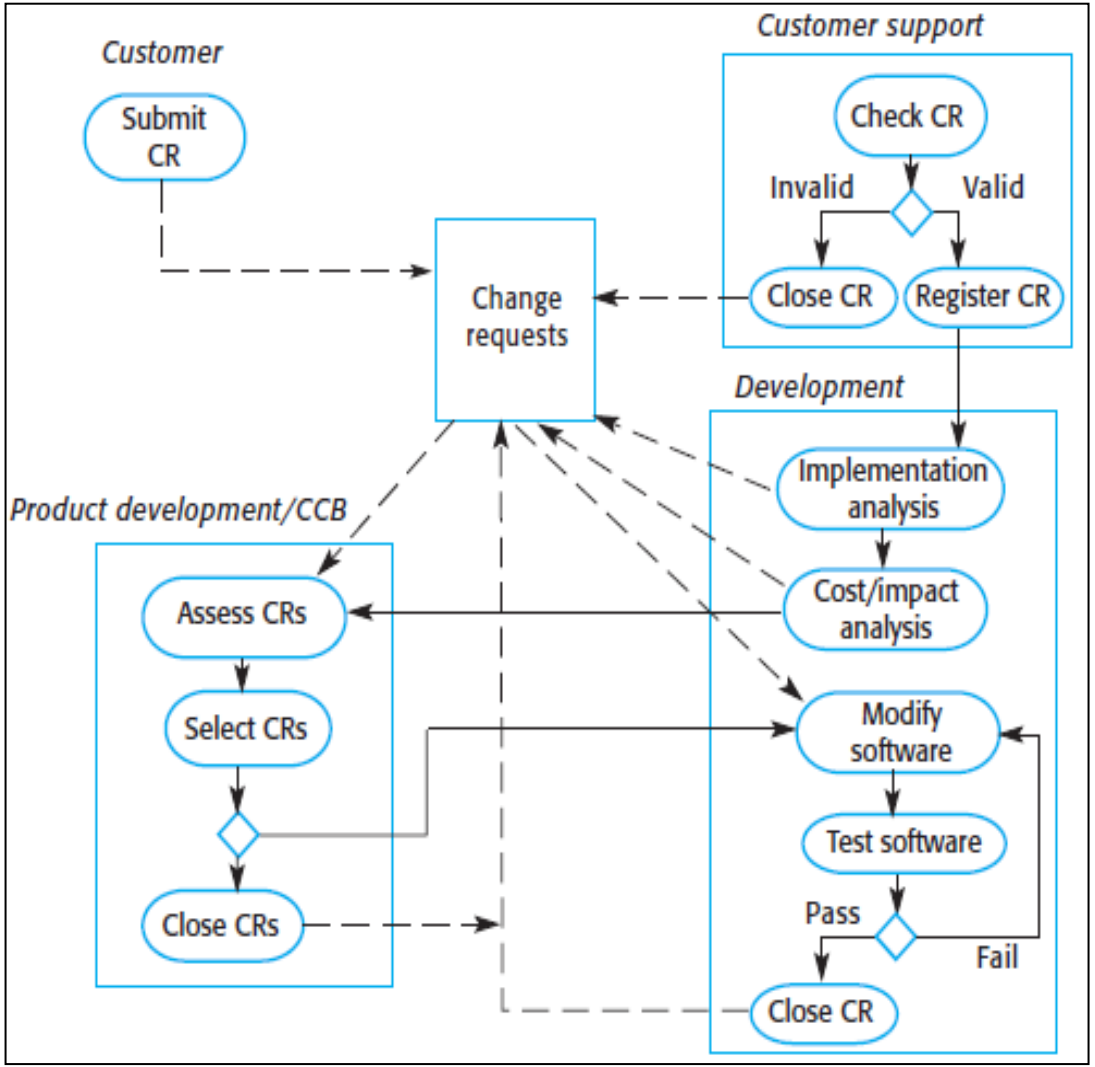
- Creates ***complete***, ***executable*** versions of the system.
- Developers upload their code to the server, which stores the official system versions.

## 3. The Target Environment

- Platform on which the system executes.
- For **real-time and embedded systems**, it is usually smaller and simpler than the development setup.
- For **large systems**, it may include databases and other apps that can't run on development machines.

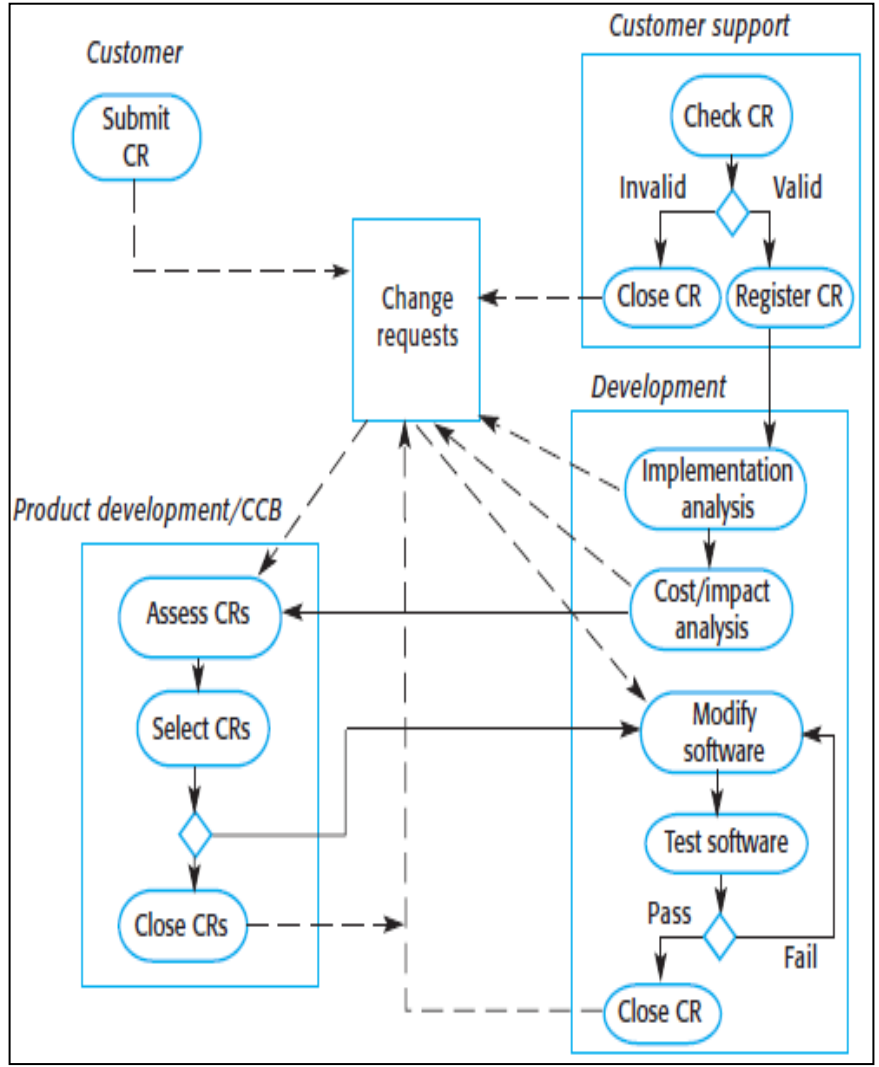
## 3. CHANGE MANAGEMENT

- Change management is the process of
  - ***Analyzing the costs and benefits of proposed changes.***
  - ***Approving changes that are cost-effective.***
  - ***Tracking which system components have been changed.***
- It is used when the software is released to customers or deployed in an organization.



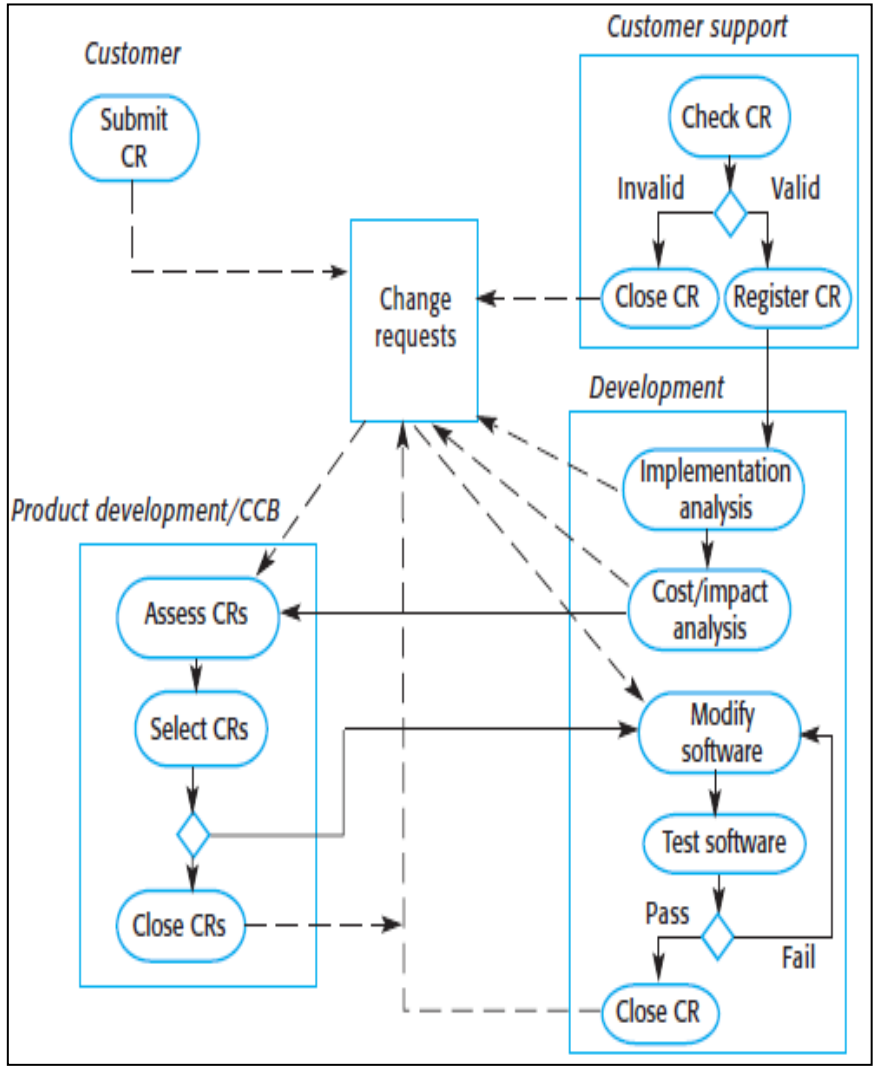
The **Change Management Process** starts when a stakeholder submits a ***Change Request Form (CRF)*** describing the needed change.

- This could be a ***bug report*** or a ***request for new features***.
- **Stakeholders** include ***owners***, ***users***, ***testers***, ***developers***, or the ***marketing team***.
- The CRF records details like the change description, recommendations, estimated costs, and key dates (requested, approved, implemented, and validated).
- Developers may also describe how the change will be made.
- As the CRF moves through the process, updates are added, showing the current status of the request.

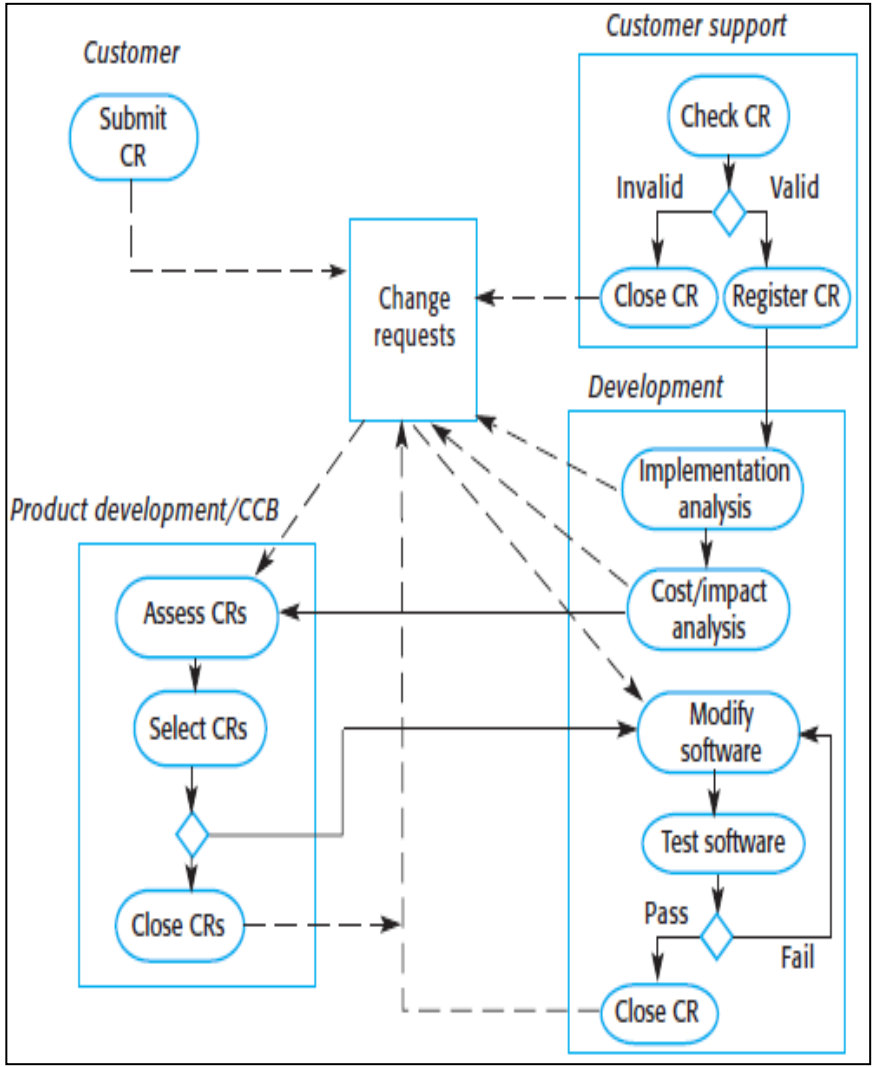


- **After a change request has been submitted, it is checked to ensure that it is valid.**
- The **checker** can be from the ***customer***, ***support team***, or the ***development team*** for internal requests.
- The request may be rejected if the bug is already reported and fixed.
- If rejected, the form is updated with the reason for closure.
- If valid, it is logged as a pending request for further analysis.

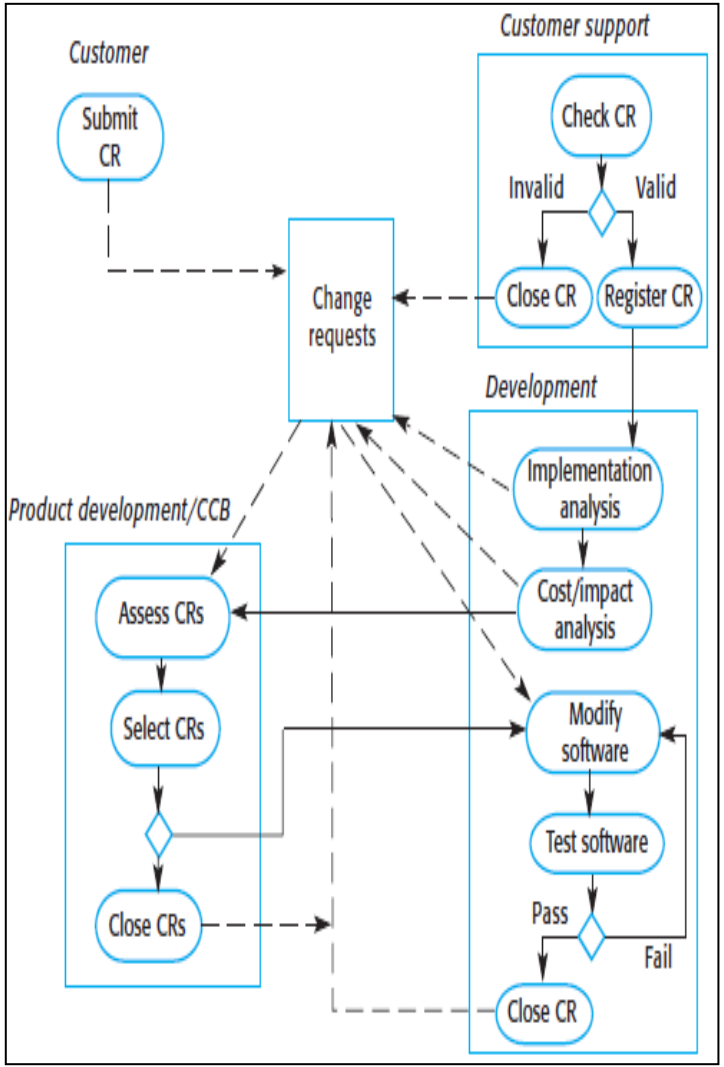
***Sometimes, what people believe to be problems are actually misunderstandings of what the system is expected to do. On occasions, people request features that have already been implemented but that they don't know about.***



- **For valid change requests, the next stage of the process is change assessment and costing.**
- The **development or maintenance team** handles the change, as they know what is involved.
- They ***check the impact*** on the rest of the system.
- All ***affected components*** are identified.
- If the change affects other parts, it will increase the cost.
- The team assesses the changes needed for system modules.
- Finally, they ***estimate the total cost***, including related component changes.



- **Following this analysis, *a separate group* decides if it is cost-effective for the business to make the change to the software.**
- For **military and government systems**, the **Change Control Board (CCB)** handles change requests.
- In **industry**, this group is called the **Product Development Group**.
- They ***review and approve*** major changes, while **small fixes** (e.g., screens or documents) are sent directly to the development team.
- The CCB looks at changes from a business perspective, checking if they are cost-effective and prioritizing them.
- **Accepted changes** go to the development team for implementation.
- **Rejected requests** are closed with no further action.



- The factors that influence the decision on whether or not to implement a change include:

**1. *The consequences of not making the change***

- When assessing a change request, you have to consider what will happen if the change is not implemented.
- If the ***change is associated with a reported system failure***, the seriousness of that failure has to be taken into account.
  - If the system failure causes the system to crash, this is very serious, and failure to make the change may disrupt the operational use of the system.
- If the failure ***has a minor effect***, such as incorrect colors on a display, then it is not important to fix the problem quickly.
- The change should therefore have a low priority.

**2. *The benefits of the change***

- Will the change benefit many users of the system, or will it only benefit the change proposer?

### 3. *The number of users affected by the change*

- If only a few users are affected, then the change may be assigned a low priority.
- Actually, changing something might not be a good idea if it means most users of the system have to adjust to it.

### 4. *The costs of making the change*

- If the change affects many parts of the system or takes too long to implement, it may be rejected.

### 5. *The product release cycle*

- If a new version was just released, the change may be delayed until the next update.

- A partially completed change request form

**Change Request Form**

**Project:** SICSA/AppProcessing **Number:** 23/02
**Change requester:** I. Sommerville **Date:** 20/07/12
**Requested change:** The status of applicants (rejected, accepted, etc.) should be shown visually in the displayed list of applicants.

**Change analyzer:** R. Looek **Analysis date:** 25/07/12
**Components affected:** ApplicantListDisplay, StatusUpdater

**Associated components:** StudentDatabase

**Change assessment:** Relatively simple to implement by changing the display color according to status. A table must be added to relate status to colors. No changes to associated components are required.

**Change priority:** Medium
**Change implementation:**
**Estimated effort:** 2 hours
**Date to SGA app. team:** 28/07/12 **CCB decision date:** 30/07/12
**Decision:** Accept change. Change to be implemented in Release 1.2
**Change implementor:** **Date of change:**
**Date submitted to QM:** **QM decision:**
**Date submitted to CM:**
**Comments:**

## 4. RELEASE MANAGEMENT

- A system release is a version of a software system that is ***distributed to customers***.
- For mass-market software, it is usually possible to identify two types of release:
  - (i) **Major releases :** Add new features
  - (ii) **Minor releases:** ***Repair bugs and fix customer problems*** that have been reported
- **Major releases** are important for software vendors because customers usually ***pay*** for them.
- **Minor releases** are usually distributed ***free*** of charge.
- A software product release is not just the executable code of the system.
- The release may also include:
  - ***Configuration files defining how the release should be configured for particular installations***
  - ***Data files, such as error messages in different languages***
  - ***An installation program that is used to help install the system on target hardware***
  - ***Electronic and paper documentation describing the system***
  - ***Packaging and associated publicity that have been designed for that release.***

# Mobile App

## 1.Major Release:

1. Adds new features, such as ***dark mode*** or ***voice commands***.
2. Customers **pay** for this upgrade.

## 2.Minor Release:

1. Fixes bugs like ***app crashes*** or ***slow loading***.
2. It is usually **free**.

## 3.Release Package Includes:

1. **Configuration files**: Settings for different phones (e.g., Android, iOS).
2. **Data files:** Error messages in different languages.
3. **Installation program:** Helps install the app on devices.
4. **Documentation:** User guide or help section.
5. **Packaging and promotion:** App store description and ads.

- The various **technical and organizational factors** should be considered when deciding on when to release a new version of a software product

| Factor | Description |
|---|---|
| ***Competition*** | • For mass-market software, a new release may be needed if competitors add new features, to avoid losing customers.<br>***A messaging app adds video calling because its competitor offers it.*** |
| ***Marketing Requirements*** | • The marketing team may set a release date.<br>• To boost upgrades, they may add new features to attract users.<br>***A phone company adds a better camera feature in its new model to convince users to upgrade.*** |
| ***Platform Changes*** | • A new software release may be needed when a new OS version comes out.<br>***An antivirus app releases an update to support the latest Windows version.*** |
| ***Technical Quality of the System*** | • If serious faults affect many customers, a new release is needed<br>• Minor faults can be fixed with patches sent online.<br>***A game releases a new version to fix major crashes. For small glitches, it sends an online patch.*** |

- **Release creation** is the ***process of creating the collection of files and documentation*** that include all components of the system release.
- This process involves several steps:

**1. Identify and Tag Code:**

- Mark the app's code and data files with a release ID in the version control system.

  ***Tag version 1.5 with new editing features.***

**2. Write Configuration Descriptions:**

- Create setup instructions for different OS.

  ***One guide for Windows, another for macOS.***

**3. Update Customer Instructions:**

- Write user guides explaining new features.

  ***Add steps for using a new filter tool.***

**4. Create Installation Scripts:**

- Write scripts to automate the installation.

  ***A script to install the app with default settings.***

**5. Design Web Pages:**

- Make web pages with release details and documentation links.

  ***A page showing new features and download links.***

**6. Prepare Master Image:**

- Create the final version for distribution.

  ***The installer file sent to app stores or customers.***

- When planning the installation of new system releases, you cannot assume that customers will always install new system releases.
- Some system users may be happy with an existing system and may not consider it worthwhile to absorb the cost of changing to a new release.
- New releases of the system cannot, therefore, rely on the installation of previous releases.
- To illustrate this problem, consider the following scenario:

1. Release 1 of a system is distributed and put into use.

2. Release 2 requires the installation of new data files, but some customers do not need the facilities of release 2 and so remain with release 1.

3. Release 3 requires the data files installed in release 2 and has no new data files of its own.